MultiQT notes

November 12, 2026

1 Notation

Unless stated otherwise, fix one disease, location and forecast horizon.

Forecasts. The index t labels a forecast round (reference week), and y_t is the outcome for its target week. Use s for an earlier round. At quantile levels $\mathcal{A} = \{\alpha_1 < \cdots < \alpha_m\} \subset (0,1)$, b_t^α is the base forecast and q_t^α the recalibrated forecast. The vectors $b_t, q_t \in \mathbb{R}^m$ collect all levels. Values are hospital counts unless a transformation is shown.

Coverage and loss. Write $\mathbf{1}\{\cdot\}$ for an indicator and $x_+ = \max(x, 0)$. Coverage at level α is $\mathrm{cov}_t^\alpha = \mathbf{1}\{y_t \le q_t^\alpha\}$, with target α. Quantile loss, with prediction z first and observation r second, is

$$\rho_\alpha(z, r) = \alpha(r - z)_+ + (1 - \alpha)(z - r)_+. \tag{1}$$

Ordering and states. Forecasts must be nondecreasing across levels. Write

$$\mathcal{K} = \{q \in \mathbb{R}^m : q^{\alpha_1} \le \cdots \le q^{\alpha_m}\},$$

and assume $b_t \in \mathcal{K}$. Projection onto a nonempty closed convex set C means the nearest point in Euclidean distance:

$$\Pi_C(x) = \arg\min_{z \in C} \|z - x\|^2.$$

A tilde marks hidden accumulated state; the untilded state is its projected version. Standard offsets use $\tilde{\theta}_t, \theta_t$; studentized offsets use $\tilde{u}_t, u_t$. Superscripts $\mathrm{std}, \mathrm{stu}$ distinguish these methods, and η or η_t is the step size.

The methodological recursions forecast, observe y_t, then update. Delayed reports and revisions are introduced in Section 3.2; implementation, weighting and evaluation symbols are defined where they are used.

2 Methodological notes

2.1 Studentized score

Standard MultiQT tracks additive residual corrections. Studentization tracks scaled residuals, so changing the scale also changes the accumulated correction in outcome units. A falling scale can shrink a positive correction after a peak without erasing the hidden state. The quantile coverage target is unchanged; this construction alone does not establish calibration guarantees.

2.1.1 Standard MultiQT and score tracking

Starting with zero hidden state, MultiQT [1] projects the forecast but updates the unprojected state:

$$\theta_t = \Pi_{\mathcal{K}-b_t}(\tilde{\theta}_t), \qquad q_t^{\text{std}} = b_t + \theta_t, \tag{2}$$

$$\text{cov}_t^{\text{std},\alpha} = \mathbf{1}\{y_t \le q_t^{\text{std},\alpha}\}, \qquad \tilde{\theta}_{t+1}^{\alpha} = \tilde{\theta}_t^{\alpha} - \eta(\text{cov}_t^{\text{std},\alpha} - \alpha). \tag{3}$$

For a score $S_t(y)$, let $r_t = S_t(y_t)$ be its observed value, c_t its threshold and $\delta = 1 - \alpha$ the target miscoverage. The scalar score tracker of Angelopoulos et al. [2] uses

$$\mathcal{P}_t = \{y : S_t(y) \le c_t\}, \qquad \text{err}_t = \mathbf{1}\{r_t > c_t\}, \tag{4}$$

and updates

$$c_{t+1} = c_t + \eta(\text{err}_t - \delta) = c_t - \eta(\mathbf{1}\{r_t \le c_t\} - \alpha). \tag{5}$$

This is a quantile-loss subgradient step. In the threshold argument,

$$\partial_z \rho_\alpha(z, r) = \begin{cases} \{-\alpha\}, & z < r, \\ [-\alpha, 1-\alpha], & z = r, \\ \{1-\alpha\}, & z > r, \end{cases} \tag{6}$$

and the selected subgradient, including ties, is

$$g_\alpha(z; r) = \mathbf{1}\{r \le z\} - \alpha \;\in\; \partial_z \rho_\alpha(z, r). \tag{7}$$

Loss depends on distance from the threshold; the update depends only on which side contains the observation.

2.1.2 Residual coordinates

The signed residual score recovers standard MultiQT:

$$S_t^{\text{std},\alpha}(y) = y - b_t^{\alpha}, \qquad r_t^{\text{std},\alpha} = y_t - b_t^{\alpha}. \tag{8}$$

Its threshold θ_t^{α} defines the forecast endpoint and coverage:

$$\{y : S_t^{\text{std},\alpha}(y) \le \theta_t^{\alpha}\} = (-\infty, b_t^{\alpha} + \theta_t^{\alpha}] = (-\infty, q_t^{\text{std},\alpha}], \tag{9}$$

$$\text{cov}_t^{\text{std},\alpha} = \mathbf{1}\{r_t^{\text{std},\alpha} \le \theta_t^{\alpha}\}. \tag{10}$$

Thus

$$\tilde{\theta}_{t+1}^{\alpha} = \tilde{\theta}_t^{\alpha} - \eta(\mathbf{1}\{r_t^{\text{std},\alpha} \le \theta_t^{\alpha}\} - \alpha), \tag{11}$$

with the same loss in residual coordinates:

$$\ell_t^{\text{std}}(\theta) = \sum_{\alpha \in \mathcal{A}} \rho_\alpha(b_t^{\alpha} + \theta^{\alpha}, y_t) = \sum_{\alpha \in \mathcal{A}} \rho_\alpha(\theta^{\alpha}, r_t^{\text{std},\alpha}). \tag{12}$$

For one level there is no ordering constraint, so this is scalar quantile tracking. For several levels, evaluate the subgradient at the projected state and apply it to the hidden state.

2.1.3 Studentized residuals and their objective

Choose finite, positive scales σ_t^α from information available at the forecast cutoff and hold them fixed when differentiating. Offsets u_t^α are in studentized units, whereas θ_t^α is in outcome units. Define

$$S_t^{\mathrm{stu},\alpha}(y) = \frac{y - b_t^\alpha}{\sigma_t^\alpha}, \qquad r_t^{\mathrm{stu},\alpha} = \frac{y_t - b_t^\alpha}{\sigma_t^\alpha}. \tag{13}$$

The score is increasing, so inversion preserves the coverage event:

$$q_t^{\mathrm{stu},\alpha} = b_t^\alpha + \sigma_t^\alpha u_t^\alpha, \qquad \mathrm{cov}_t^{\mathrm{stu},\alpha} = \mathbf{1}\{r_t^{\mathrm{stu},\alpha} \le u_t^\alpha\}. \tag{14}$$

Quantile loss of the standardized scores is a weighted outcome-unit loss:

$$\ell_t^{\mathrm{stu}}(u) = \sum_{\alpha\in\mathcal{A}} \rho_\alpha(u^\alpha, r_t^{\mathrm{stu},\alpha}) = \sum_{\alpha\in\mathcal{A}} \frac{\rho_\alpha(b_t^\alpha + \sigma_t^\alpha u^\alpha, y_t)}{\sigma_t^\alpha}. \tag{15}$$

Its selected subgradient is

$$g_t^{\mathrm{stu},\alpha}(u_t) = \mathbf{1}\{r_t^{\mathrm{stu},\alpha} \le u_t^\alpha\} - \alpha = \mathrm{cov}_t^{\mathrm{stu},\alpha} - \alpha. \tag{16}$$

Indeed, away from ties the inverse scale in the loss cancels the scale from differentiation of the forecast:

$$\frac{\partial}{\partial u^\alpha} \frac{\rho_\alpha(b_t^\alpha + \sigma_t^\alpha u^\alpha, y_t)}{\sigma_t^\alpha} = \mathbf{1}\{r_t^{\mathrm{stu},\alpha} \le u^\alpha\} - \alpha. \tag{17}$$

At ties use (7). Starting from $\tilde{u}_1 = 0$, the update is

$$\tilde{u}_{t+1}^\alpha = \tilde{u}_t^\alpha - \eta g_t^{\mathrm{stu},\alpha}(u_t). \tag{18}$$

The projected threshold determines coverage; only the hidden threshold accumulates increments. This is not gradient descent on the unweighted outcome-unit loss in u: that gradient has an additional factor σ_t^α. The coding section also tests median-centred forecasts and nonzero initial states.

2.1.4 Ordering and projection

Ordering always concerns the returned forecasts. For standard residuals,

$$C_t^{\mathrm{std}} = \mathcal{K} - b_t, \qquad b_t + \theta \in \mathcal{K} \iff \theta \in C_t^{\mathrm{std}}, \tag{19}$$

$$\theta^{\alpha_i} - \theta^{\alpha_{i+1}} \le b_t^{\alpha_{i+1}} - b_t^{\alpha_i}, \qquad i = 1, \dots, m-1. \tag{20}$$

For studentized residuals, collect the scales in $A_t = \mathrm{diag}(\sigma_t^{\alpha_1}, \dots, \sigma_t^{\alpha_m})$. Then

$$C_t^{\mathrm{stu}} = \{u : b_t + A_t u \in \mathcal{K}\} = A_t^{-1}(\mathcal{K} - b_t). \tag{21}$$

This is a nonempty closed convex polyhedron containing zero, generally not a cone. Unequal scales do not preserve $\mathcal{K}$, so it cannot generally be written as $\mathcal{K} - A_t^{-1} b_t$. The unique Euclidean projection gives

$$u_t = \Pi_{C_t^{\mathrm{stu}}}(\tilde{u}_t), \qquad q_t^{\mathrm{stu}} = b_t + A_t u_t. \tag{22}$$

Changing variables to $q = b_t + A_t u$ yields weighted isotonic regression:

$$q_t^{\mathrm{stu}} = \arg\min_{q\in\mathcal{K}} \sum_{\alpha\in\mathcal{A}} \frac{(q^\alpha - b_t^\alpha - \sigma_t^\alpha \tilde{u}_t^\alpha)^2}{(\sigma_t^\alpha)^2}, \tag{23}$$

For positive-definite W, write $\Pi_{\mathcal{K}}^W(v) = \arg\min_{q\in\mathcal{K}} (q - v)^\top W (q - v)$ for weighted projection. Then the equivalent projection identity is

$$b_t + A_t \Pi_{A_t^{-1}(\mathcal{K} - b_t)}(\tilde{u}_t) = \Pi_{\mathcal{K}}^{A_t^{-2}}(b_t + A_t \tilde{u}_t). \tag{24}$$

Weights $1/(\sigma_t^\alpha)^2$ follow from Euclidean projection in score coordinates; ordinary residual coordinates give equal weights.

2.1.5 Common and constant scales

For a scale σ_t common to all levels, $A_t = \sigma_t I$, where I is the identity matrix. Positive scalar multiplication preserves $\mathcal{K}$:

$$C_t^{\text{stu}} = \mathcal{K} - b_t/\sigma_t, \qquad q_t^{\text{stu}} = \Pi_{\mathcal{K}}(b_t + \sigma_t \tilde{u}_t). \tag{25}$$

The projection is ordinary isotonic regression. This is standard MultiQT on b_t/σ_t and y_t/σ_t, with the returned forecast multiplied by σ_t. For constant σ, it is also equivalent to standard MultiQT on the original inputs after matching initialization and converting state and rate:

$$\tilde{\theta}_t = \sigma \tilde{u}_t, \qquad \eta_{\text{std}} = \sigma \eta_{\text{stu}}. \tag{26}$$

For varying σ_t, changing the scale rescales the accumulated state; changing only the current rate does not reproduce that effect.

2.2 Questions and ideas

Which property of the base forecast should another score recalibrate? Increasing, invertible scores preserve the quantile coverage event; changing the target to PIT or highest-density-region calibration would require a separate formulation.

3 Coding Updates

The tracker transform ψ is the identity, $\psi(x) = x$ (`identity`), for every method except `fixed_log` and `leaky_log`, which run standard MultiQT on $\psi(x) = \log(1 + x)$ (`log1p`). The transformed base is $\beta_t^\alpha = \psi(b_t^\alpha)$. Offsets θ_t are in these tracker units. The count divisor c_{scale} used for scoring is population/100,000 by default.

3.1 Data

Script 00 downloads CMU–TimeSeries hub forecasts and dated NHSN reports, and generates our own base forecasts where the required inputs are available. Script 01 prepares cases (one disease, location and forecast horizon) from the selected base forecasts and reports. Cases retain generation cutoffs, revisions, feedback schedules for delays 0–4, populations and filled-forecast flags.

Reference weeks are Saturdays (`reference_date`); the target week of y_t is `target_end_date` in forecasts and `target_date` in prepared cases. The information cutoff ι_t is normally the Wednesday of the reference week, or the actual generation date where known. Write $y_s^{[v]}$ for the latest report for the target week of row s, dated on or before cutoff v (archive field `version`). A valid report is finite, nonnegative and not excluded by review. "Latest counts" means the latest reports in the downloaded archive, through 30 October 2026.

Hub forecasts: the active design. The active design uses submitted CMU–TimeSeries forecasts for both diseases from reference date 23 November 2024. The stored quantiles are used subject to the cleaning and filling below. Feedback uses the report available at the relevant cutoff, while evaluation uses latest counts.

CMU–TimeSeries is an ensemble of Delphi models. Autoregressive quantile models use recent hospital admissions, sometimes with emergency-department data; linear-trend models extrapolate recent changes, and climatological models use patterns from earlier years. The ensemble takes a weighted average of the component forecasts at each quantile level. Components and weights can vary by date, location and horizon. We use these archived ensemble quantiles as b_t^α and apply our recalibration to them. See the CMU model description and the pinned ensemble code.

Target dates determine the split: tuning through 31 May 2025, then evaluation from June 2025 through September 2026. Overall scores use all available evaluation weeks; before/after-peak scores restrict these to the corresponding peak windows. Flu has long summer gaps in submitted forecasts, rather than an explicit winter-only evaluation rule. The flu winter 2024/25 peak belongs to tuning. Plotted target weeks after September 2026 are outside the evaluation period and are not scored.

Own `windowed_seasonal` forecasts: the longer-history alternative. The longer-history setup starts on reference date 14 October 2023, with tuning target dates through 27 April 2024 and evaluation after the reporting gap below, through 30 September 2026. Script 00 implements generation of both COVID and flu forecasts with unchanged, pinned Delphi code and packages and historical HHS reports for the earlier period.

`windowed_seasonal` is one autoregressive model, refitted at each reference date for each horizon. It fits quantile regression (solver `fn`, as in Delphi's production) to pooled training rows, using the latest available weekly admission count and its one-week lag. Counts undergo a fourth-root transform, centering and scaling within each location and source; predictions are transformed back to nonnegative counts and their quantiles sorted. For flu, the training window selects weeks near the corresponding time of year across available years. The COVID configuration uses no seasonal restriction despite the model name. These forecasts then serve as

the base for our recalibration. Script 00 generates horizons $0, \ldots, 3$ only. At $h = -1$ the target week is usually already reported, so the outcome equals the latest-count predictor: the fit is exact and all quantiles coincide (zero width). Delphi's submissions also take $h = -1$ from other models. Without it, the nowcast and revision indicators (Section **??**) are unavailable. See the pinned model implementation and COVID/flu settings.

Inputs for own flu forecasts. Flu adds historical ILI+ and adjusted FluSurv-NET training rows to the hospital counts; COVID uses hospital counts alone. The eight required CSVs have been downloaded in `aux_data.zip`: `flu_burden.csv`, `us_pop.csv` and six WHO/NREVSS laboratory files. Script 00 now extracts them to `external/delphi/aux_data/flusion_data/` and supplies them to the Delphi functions before forecast generation.

How earlier reports are reconstructed. Our NHSN archive contains historical counts, but report versions start on 19 November 2024. Script 00 now reconstructs earlier weekly HHS versions with the pinned Delphi aggregation script. For a Saturday target date, it sums admission days from the preceding Saturday through Friday; a weekly count is available only when all seven daily counts are present. The HHS versions precede the first NHSN version, which supersedes them. Base forecasts use reports available at the generation date. Script 01 uses HHS versions for earlier feedback where available, including later revisions in the re-run design; scoring uses latest NHSN counts.

Other limitations of the longer-history experiment. These are not flu-specific. Target weeks 4 May–2 November 2024 have incomplete reporting and are excluded from learning and scoring; our forecast schedule pauses across this gap. Stateful methods carry accumulated corrections and weights forward when rounds resume, rather than restarting. This reporting gap and differences between the earlier HHS and later NHSN sources affect both diseases. With four horizons instead of five, results pooled over horizons are not directly comparable with those of the hub forecasts.

Cleaning and filling.

1. **Missing forecasts.** Fill gaps of at most six rounds from the most recent earlier forecast for the same target: horizon h at reference date r uses $h + k$ at $r - 7k$ days, with the smallest available $k > 0$. Horizon 3 has no source. Filled rows are flagged and scored; unfillable rounds remain absent.
2. **Broken submissions.** Remove the known Delphi bugs: COVID 23 November 2024; COVID $h = -1$ on 25 April 2026; US flu 14 and 21 December 2024 and 4 January 2025. Also remove all-zero submissions across every location and quantile of a disease–round–horizon (flu $h = -1$, 22 November 2025).
3. **Missing reports.** If unpublished or currently invalid, stateful feedback waits and re-runs omit it; neither substitutes an older valid report. Keep the forecast timeline when latest truth is unavailable, but omit that target from scoring; earlier valid feedback can still be used. Targets 4 May–2 November 2024 are excluded from learning and scoring.
4. **Report errors.** A reviewed exclusion table masks identified erroneous report versions, retaining later corrections and their target weeks. Each exclusion records its evidence; decisions based on later revisions are retrospective cleaning. For example, Pennsylvania COVID for 16 November 2024 changed from 6,124 to 1,005 the next day.

Disease	Missing reference weeks	Unfilled target weeks, $h = 2$
COVID	25 January–1 February 2025	15 February 2025
Both	25 October–15 November 2025	15–29 November 2025
Flu	25 January 2025; 18 April 2026	None
Flu	7 June–11 October 2025	21 June–25 October 2025
Flu	6 June–3 October 2026	20 June–17 October 2026

The COVID $h = 2$ weight plot (`multiqt_weight_timelines/covid_horizon2_delay0.pdf`, p. 1) shows the first two gaps; only the first missing round can use an earlier $h = 3$ forecast. US flu also lacks target 4 January 2025 at $h = 2$ after successive broken submissions. Lighter bars denote filled rows. Lines break at gaps longer than 28 days and can bridge shorter missing intervals. Updates occur only at retained rounds: pending feedback waits across unfilled rounds, without resetting the state.

Reporting pause in autumn 2025. NHSN published no report between 26 September and 14 November 2025, a pause that coincides with the US federal government shutdown (1 October–12 November). Target weeks 27 September–8 November were first reported together on 14 November, 6 to 48 days after the week ended; weekly reports resumed on 19 November. Both diseases are affected. CMU kept submitting through 18 October and then skipped four rounds. Each round of 2025 around the pause, with its cutoff on the Wednesday before:

Round	Newest NHSN report at the cutoff	CMU forecast	Learned at delay 0
27 Sep	24 Sep, through week 20 Sep	submitted	week 20 Sep
4 Oct	26 Sep (revision)	submitted, no new counts	—
11 Oct	26 Sep	submitted, no new counts	—
18 Oct	26 Sep	submitted, no new counts	—
25 Oct	26 Sep	missing; filled for $h \leq 2$	—
1 Nov	26 Sep	missing; filled for $h \leq 1$	—
8 Nov	26 Sep	missing; filled for $h \leq 0$	—
15 Nov	26 Sep	missing; filled for $h = -1$	—
22 Nov	19 Nov, through week 15 Nov	submitted	weeks 27 Sep–15 Nov
29 Nov	26 Nov, through week 22 Nov	submitted	week 22 Nov

The trackers learn nothing for seven rounds: at delay 0 the last update before the pause is in the round of 27 September and the next in that of 22 November (delay 1: 4 October and 29 November; the re-run learns from the same reports). The forecasts of 4–25 October carry the corrections of late September, and the weeks of the pause are then learned in one step. CMU's forecasts of 4–18 October had no new NHSN counts, so they extrapolate one to three weeks further than their horizon says. The pause came right after the summer COVID wave: 45 of the 52 locations without the US have a COVID peak in August–September 2025 (the US on 6 September, 11,048 admissions, falling to 3,649 by 25 October), and 287 of the 800 COVID after-peak weeks (36%) are among the weeks first reported on 14 November. The COVID results after peaks thus partly measure trackers that could not see the decline. No flu peak is near the pause.

Locations. Every disease–location–horizon case learns independently. Detailed plots show the US, California, Texas, New York and Pennsylvania. Scores cover all 53 locations. Tuning and pooled summaries use the 52 non-US locations; the US is evaluated separately with their selected parameters. Forecasts are never averaged across locations.

3.2 Feedback

Feedback uses the latest report at each cutoff as defined in Section 3.1, independently of the recalibration method. Retrospective initialization is described in Section 3.3.

Stateful learning. Let ν_s be the publication date of the first valid report for row s. Wait d calendar weeks, giving maturity date $m_s^{(d)}$, then consume the newest report when valid at the next retained forecast cutoff with $t > s$. Write $\tau_d(s)$ for that round and $\mathcal{R}_t^{(d)}$ for the rows consumed before forecasting at t:

$$m_s^{(d)} = \nu_s + 7d \text{ days}, \tag{27}$$

$$\tau_d(s) = \min\{t > s : \iota_t \geq m_s^{(d)},\ y_s^{[\iota_t]} \text{ valid}\}, \qquad \mathcal{R}_t^{(d)} = \{s : \tau_d(s) = t\}. \tag{28}$$

An invalid newest report postpones learning; an older version is never substituted. Use the report at actual consumption time ι_t, even if a missing forecast delayed it beyond maturity. Each row is learned at most once; subsequent revisions are ignored. The report used for a consumed row is

$$\bar{y}_s^{(d)} = y_s^{[\iota_{\tau_d(s)}]}, \qquad \tau_d(s) \text{ exists}. \tag{29}$$

Plain MultiQT on counts starts at $\tilde{\theta}_0 = 0$ and, once active, updates against previously returned forecasts before computing q_t:

$$\tilde{\theta}_t^{\alpha} = \tilde{\theta}_{t-1}^{\alpha} - \eta_t \sum_{s \in \mathcal{R}_t^{(d)}} (\mathbf{1}\{y_s^{[\iota_t]} \leq q_s^{\alpha}\} - \alpha), \tag{30}$$

$$\theta_t = \Pi_{\mathcal{K}-b_t}(\tilde{\theta}_t), \qquad q_t = b_t + \theta_t. \tag{31}$$

The hidden state is unprojected; an empty release set gives no update. The `log1p` tracker applies the same rule in ψ coordinates: ψ is increasing, so the coverage events and the update (30) are unchanged, with $\tilde{\theta}_t$ in log units, and the forecast is

$$q_t = \psi^{-1}\left[\Pi_{\mathcal{K}}(\beta_t + \tilde{\theta}_t)\right], \qquad \psi(x) = \log(1+x), \quad \psi^{-1}(z) = e^z - 1. \tag{32}$$

The projection acts in log units, before the inversion. Forecasts stay above -1; as with the `identity` tracker, they are not clipped at zero.

Weekly re-run. For forecasts and states, a superscript $[t]$ identifies the re-run at reference week t: $q_s^{[t]}$ is its reconstructed forecast for row s. This differs from the report cutoff date v in $y_s^{[v]}$. At reference week t, reconstruct plain MultiQT from zero using reports current at ι_t, retaining the release positions $j = \tau_d(s)$ and delay d, but omitting invalid current reports:

$$\tilde{\theta}_j^{\alpha,[t]} = \tilde{\theta}_{j-1}^{\alpha,[t]} - \eta_j^{[t]} \sum_{\substack{s \in \mathcal{R}_j^{(d)} \\ y_s^{[\iota_t]} \text{ valid}}} (\mathbf{1}\{y_s^{[\iota_t]} \leq q_s^{\alpha,[t]}\} - \alpha), \tag{33}$$

$$\theta_j^{[t]} = \Pi_{\mathcal{K}-b_j}(\tilde{\theta}_j^{[t]}), \qquad q_j^{[t]} = b_j + \theta_j^{[t]}. \tag{34}$$

Only $q_t^{[t]}$ is returned; earlier returned forecasts are unchanged. Reference scales follow Section 3.3. Studentized methods use (22), reconstruct their scale history and return the base until a required scale exists.

Report filtering. Select the latest report at the cutoff, then apply the reviewed exclusions; never substitute an older or future report. Count the delay from the first acceptable report. Stateful updates wait for an acceptable current report; re-runs omit excluded versions. Use the latest version for scoring and peak detection if acceptable; otherwise leave that target unscored for every method. Apply the same exclusions to the training reports of our own base forecasts.

3.3 Tuning

Reference scales use the first 20 consumed rows in release order, $I_{20}^{(d)}$, which need not span 20 calendar weeks. Parameter selection uses the tuning period; these are separate operations.

Reference scales and retrospective initialization. Write Q_p for an empirical p-quantile. The initial error scale S_0, distinct from the score function S_t, pools absolute base errors in tracker units:

$$S_0 = \max\Big\{\varepsilon_S, Q_{0.9}\Big(\Big\{|\psi(b_s^\alpha) - \psi(\bar{y}_s^{(d)})| : s \in I_{20}^{(d)},\ \alpha \in \mathcal{A}\Big\}\Big)\Big\}, \qquad \varepsilon_S = 10^{-6}. \tag{35}$$

It sets rates $\eta = \kappa S_0$, with rate multiplier κ, and studentized scale bounds $[.05S_0, S_0]$; studentized step sizes are selected in standardized units. Script 02 labels the multiplier m, distinct from $m = |\mathcal{A}|$ here. The reference loss L_0 is defined in Section 3.5.3.4.

With the default `replay_warmup=TRUE`, both constants use the eventual first 20 releases and apply from the start. Forecasts before the 20th release therefore use retrospective scale information. With replay disabled, learning waits for warmup; fewer than 20 releases leaves learning inactive. Estimate both constants from the stateful feedback schedule and keep them fixed across weekly re-runs; states, weights and time-varying scales are reconstructed. Evaluation never resets the tracker, so its initial evaluated correction is generally nonzero.

Cases and selection. Tune COVID and flu individually, each with the `log1p` tracker (`fixed` and `leaky` only) before the `identity` tracker (every tuned method). For each disease and tracker, run stateful delay 0, delay 1 and weekly re-run at delay 0. Each of these 12 configurations has one tracker-rate tuning stage, starting from defaults. Configurations without tuning rows retain the defaults. Tune all horizons of the base: $-1, \ldots, 3$ with hub forecasts, $0, \ldots, 3$ with our own base forecasts.

Candidates replay the full history with common S_0, L_0 within each case, tracker and feedback design. Only tuning target weeks are scored, including initialization rows when valid. Scores use latest archived truth, which need not have been available before evaluation. Pool case-level means equally over non-US locations and horizons. The criterion is log quantile loss for `log1p` and quantile loss per 100,000 for `identity`, without L_0 normalization.

Select tracker rates separately for `fixed`, `leaky`, `studentized`, `studentized_base`, `median_constant`, `studentized_full` and `studentized_level_rms`. `fixed_log` and `leaky_log` use the rates selected for `fixed` and `leaky` with the `log1p` tracker, that is by log quantile loss.

Let $\overline{\mathcal{L}}_j(\vartheta)$ be the mean loss of method j at candidate ϑ over tuning cases, and let $\overline{\mathrm{CE}}_j(\vartheta)$ be its mean calibration error. Subscript 0 denotes the original forecast. For each method, minimize the sum of percentage changes from the original:

$$E_j(\vartheta) = 100\left[\frac{\overline{\mathcal{L}}_j(\vartheta)}{\overline{\mathcal{L}}_0} - 1\right] + 100\left[\frac{\overline{\mathrm{CE}}_j(\vartheta)}{\overline{\mathrm{CE}}_0} - 1\right]. \tag{36}$$

The code calls this the elbow rule: one percentage point of loss change and one percentage point of calibration-error change have equal weight. It does not estimate curvature. Ties select the smaller parameter value. Each method has one tuned rate, so its grid is evaluated separately; there is no Cartesian search.

Setting	Candidates
`fixed_multiplier`	.001, .0025, .005, .01, .02, .05, .1, .2, .4
`median_rate`	.003, .01, .03, .1, .3, 1
`leaky_multiplier`	.001, .002, .004, .008, .02, .05, .1, .2, .3, .5, 1

Multipliers scale S_0. `median_rate` is standardized.

Fixed and inherited settings. All learned mixtures use fixed $H = 8$ processed weight updates; sharing is therefore $\gamma = 1/(2H) = 1/16$ where enabled. Learned mixtures use adaptive λ from $\lambda_{\text{start}} = 2.72$; $\lambda = 0$ gives equal weights in `exp_weights()`. Recent-start count is 8; the two-rate recent starts use $.03S_0$ and $.3S_0$; the outer rate is $.001S_0$. Other constants remain as specified with each method. `studentized_zero` and `median_constant_zero` inherit their corresponding normal-start rates, and outer variants retain $H = 8$ of their inner mixtures.

Tuning plots. The current `multiqt_tuning.pdf` pairs calibration error with quantile loss, both as percentage changes from the original. Solid/dashed lines use the log/identity criteria. Symbol size grows with the candidate value; circles mark selected values and an open square at $(0, 0)$ marks the original. A dotted line of slope -1 through each selected point shows its value of (36); no candidate from that curve lies below it. Only pooled pages by disease and feedback design are produced, using the non-US locations and tuning horizons above. The `on_grid_edge` flag marks an endpoint selection, not evidence of an interior optimum.

3.4 Evaluation

Peaks and evaluation windows. Let $y_t^\star$ be the latest archived count for the target of row t, and $y^\star(D)$ that of target week D, so $y_t^\star = y^\star(T_t)$ for the target date T_t of row t. Peaks are found per disease and location. Script 01 omits invalid latest counts, including reviewed exclusions, and the incomplete-reporting weeks 4 May–2 November 2024. Write $D_1 < \cdots < D_n$ for the retained target weeks, $x_i = y^\star(D_i)$ for their counts and $\mathcal{N}_i(k) = \{j \in \{1, \ldots, n\} : |j - i| \le k\}$ for the retained observations within k of i. Indices count retained observations, not calendar weeks: across the omitted weeks, neighbours D_i, D_{i+1} can be months apart. `find_peaks()` then uses three steps:

1. *Smoothing and candidates.* The centred three-observation mean, with the endpoints unchanged, is
$$\mu_i = \tfrac{1}{3}(x_{i-1} + x_i + x_{i+1}), \quad 1 < i < n, \qquad \mu_1 = x_1, \quad \mu_n = x_n. \tag{37}$$
Observation i is a candidate if μ_i is the largest smoothed value within eight observations on either side, the earliest at ties:
$$\mu_i \ge \mu_j \text{ for all } j \in \mathcal{N}_i(8), \qquad \mu_i > \mu_j \text{ for all } j \in \mathcal{N}_i(8) \text{ with } j < i. \tag{38}$$

2. *Prominence.* On each side, search up to the nearest strictly higher smoothed value, or the series end:
$$l_i = \max(\{1\} \cup \{j < i : \mu_j > \mu_i\}), \qquad r_i = \min(\{n\} \cup \{j > i : \mu_j > \mu_i\}). \tag{39}$$
The prominence is the smoothed height above the higher of the two minima in these ranges,
$$\pi_i = \mu_i - \max\Big\{\min_{l_i \le j \le i} \mu_j,\ \min_{i \le j \le r_i} \mu_j\Big\} \ge 0, \tag{40}$$
so $\pi_1 = \pi_n = 0$. A candidate is kept if
$$\pi_i > 0, \qquad \pi_i \ge 0.3\,\mu_i, \qquad \pi_i \ge 0.1\,M, \tag{41}$$
where $M = \max\{x_j : D_j \text{ is a forecast target week}\}$, over the forecast target weeks of all horizons. Equivalently, on both sides the smoothed counts fall to $\min\{0.7\,\mu_i,\ \mu_i - 0.1M\}$ or below before they exceed μ_i or the series ends. Script 01 stores π_i/μ_i as `relative_prominence`.

3. *Placement.* A kept candidate i moves to the largest raw count within one observation, the earliest at ties:
$$i' = \min\Big\{\arg\max_{j \in \mathcal{N}_i(1)} x_j\Big\}. \tag{42}$$
Its peak date $D_{i'}$ is kept if it lies between the first and the last forecast target week.

Peaks are hindsight labels, used for prediction only by the target oracle.

Let $p(t)$ be the peak date $D_{i'}$ nearest to T_t (earlier at ties), and $(T_t - p(t))/7$ their distance in calendar weeks. For evaluation rows E with valid latest truth, score the windows

$$\begin{aligned} W_{\text{overall}} &= E, \\ W_{\text{before}} &= \{t \in E : -7 \le (T_t - p(t))/7 \le 0\}, \\ W_{\text{after}} &= \{t \in E : 1 \le (T_t - p(t))/7 \le 8\}. \end{aligned} \tag{43}$$

Rows count once; the peak week belongs to "up to peak". Overall also includes weeks outside these windows. Missing rows and period boundaries shorten windows without discarding peaks. Period membership follows the target date; its nearest peak may lie in another period. The main plots use these windows over the full evaluation period.

Loss and calibration error. A case fixes location, horizon and feedback design within a disease. For a selected window W of $N = |W|$ rows, average over rows and all 23 quantile levels:

$$\mathcal{L}^{\text{nat}}(W) = \frac{1}{mN} \sum_{t \in W} \sum_{\alpha \in \mathcal{A}} \rho_\alpha(q_t^\alpha / c_{\text{scale}}, y_t^\star / c_{\text{scale}}), \tag{44}$$

$$\mathcal{L}^{\log}(W) = \frac{1}{mN} \sum_{t \in W} \sum_{\alpha \in \mathcal{A}} \rho_\alpha(\log(1 + \max\{q_t^\alpha, 0\}), \log(1 + y_t^\star)), \tag{45}$$

$$\text{CE}(W) = \frac{1}{m} \sum_{\alpha \in \mathcal{A}} \left| \frac{1}{N} \sum_{t \in W} \mathbf{1}\{y_t^\star \le q_t^\alpha\} - \alpha \right|. \tag{46}$$

Columns are `quantile_loss`, `log_quantile_loss` and `coverage_error`; tuning uses `loss`, `log_loss` and `coverage_error`. CE averages weeks before taking the absolute residual. All feedback designs use the same latest truth; empty windows give missing scores. Evaluation losses are never divided by L_0. Scripts 04/06 display the criterion of the tracker; script 02 displays loss per 100,000.

All-location trade-offs. Write $\mathcal{L}_{j,c}$ and $\text{CE}_{j,c}$ for the mean quantile loss and calibration error of method j in case c; $j = 0$ is the original. A set $\mathcal{C}$ contains J cases. In `multiqt_peak_reasoning_tradeoff.pdf`, columns are horizons $-1, \ldots, 3$; blocks are delay 0, delay 1 and re-run, each showing overall, up to peak and after peak. Lower left is better. The first page per disease shows individual non-US cases and method means

$$\left(J^{-1} \sum_c \mathcal{L}_{j,c},\ J^{-1} \sum_c \text{CE}_{j,c} \right).$$

These average location scores, not forecasts or pooled coverage indicators. The first row of `multiqt_error_tradeoff.pdf` uses the same aggregation for fixed rates. Feedback designs have separately selected parameters, so their differences include retuning. Axes vary by panel; all-location limits use the 1st–99th percentiles and mean points, with loss capped near four times the base. Extremes can be hidden or marked `>`. Original/common-weight/outer methods use open circles/triangles/squares or diamonds; exact forecast duplicates are hidden.

Evaluation tree. Script 06 splits pooled disease cases by feedback design, then horizon. For each window it reports

$$R_j(\mathcal{C}) = 100 \left[1 - \frac{\sum_{c \in \mathcal{C}} N_c \mathcal{L}_{j,c}}{\sum_{c \in \mathcal{C}} N_c \mathcal{L}_{0,c}} \right], \tag{47}$$

$$P_j(\mathcal{C}) = J^{-1} \sum_{c \in \mathcal{C}} \mathbf{1}\{\mathcal{L}_{j,c} < \mathcal{L}_{0,c}\}. \tag{48}$$

`reduction` is a ratio of week-weighted losses (positive is better); `share_improved` weights cases equally, counts ties as not improved, and is printed for at least five cases. CE panels also average case scores with week weights. Each page displays the ten methods with lowest pooled after-peak loss: a descriptive evaluation ranking, not tuning. Tree panels share axes; clipping is marked `<` or `>`. By contrast, script 02 phase-rate plots show loss change $100(\mathcal{L}_j/\mathcal{L}_0 - 1)$, where negative is better.

Forecast curves. Files in `multiqt_method_forecasts/` show levels $.1, \ldots, .9$ and weekly loss per 100,000 at a fixed horizon; gray marks tuning, dotted lines peaks. Files in `multiqt_season_forecasts/` add coverage and loss trajectories for selected fixed rates. `multiqt_base_forecast_paths.pdf` shows full count timelines, median paths around all retained peaks and three diagnostic indicators (Section **??**). It spans tuning and evaluation; its flu winter 2024/25 peak is within tuning, not held-out evidence. Weight timelines show how each mixture combines its experts. Gap dates are listed in Section 3.1.

3.5 Methods

The methods below use the preceding data, feedback, tuning and evaluation conventions unless stated otherwise.

3.5.1 Single trackers

These methods maintain one MultiQT tracker for each case.

Fixed rate: `fixed`. Use (30)–(31) with $\eta = \kappa S_0$ in tracker units. Tune κ separately (default .05). Corrections persist across rounds.

Leaky tracker: `leaky`. Use Delphi's leak (`fast_decay` of their tracker implementation [4]): Fixed MultiQT whose hidden state loses the fraction $\delta = .1$ at every processed reference week, before its update,

$$\tilde{\theta}_t^{\alpha} = (1-\delta)\tilde{\theta}_{t-1}^{\alpha} - \eta_t \sum_{s \in \mathcal{R}_t^{(d)}} \left(\mathbf{1}\{y_s^{[\iota_t]} \le q_s^{\alpha}\} - \alpha\right), \qquad \eta_t = \kappa S_0, \tag{49}$$

with the forecast of (31). A correction that new feedback does not renew fades back to the base, with a half-life of about 6.6 reference weeks; the decay applies even without feedback, and gaps do not compound it. This is the fast term of `ref_op` without its slow term and rolling rate. Tune κ separately; δ is Delphi's value and is not tuned. The method also belongs to the score bank.

Log1p trackers: `fixed_log` and `leaky_log`. Fixed and Leaky MultiQT on the `log1p` tracker, $\psi(x) = \log(1+x)$: the updates (30) and (49) of the hidden state $\tilde{\theta}_t$, in log units, and the forecast (32). The rate is $\eta = \kappa S_0$ with S_0 from (35) in log units (in a re-run, frozen from the stateful design, as for the other methods), and κ is the rate selected for `fixed` and `leaky` with the `log1p` tracker. Their correction is multiplicative, (55): they compare an additive correction with one that scales with the level (Section 3.5.2).

Persistent restart: `restart_persistent`. Keep the first eligible weekly-start expert with $\kappa = .05$ (Section 3.5.3.5). It learns only from forecasts made since its start. Its rate is not separately tuned, so it need not reproduce `fixed`.

3.5.2 Studentized scores

Studentized variants and median initialization. Studentized methods track counts. Use $a_t^\alpha = b_t^\alpha$ for the full base or $a_t^\alpha = b_t^5$ for its median. After processing available feedback, return

$$q_t = \Pi_{\mathcal{K}}^{A_t^{-2}}(a_t + A_t\tilde{u}_t). \tag{50}$$

For a common scale, the projection is the ordinary isotonic projection. After warmup, the delayed update is applied to the hidden state,

$$\tilde{u}_t^\alpha = \tilde{u}_{t-1}^\alpha + \eta \sum_{s \in \mathcal{R}_t^{(d)}} \left(\alpha - \mathbf{1}\{\bar{y}_s^{(d)} \le q_s^\alpha\}\right). \tag{51}$$

Compare each report with its historical forecast q_s, reconstructed inside a re-run when applicable. Project only the forecast; the hidden state carries forward throughout the run. The standardized step has default $\eta = .1$ and is tuned for each normal-start or full-base method. Zero-start variants inherit the corresponding normal-start rate.

Scales from recent errors or the current forecast. Let $\mathcal{H}_t^{(d)}$ contain the last five consumed rows, or fewer if unavailable, and set $e_s^\alpha = \bar{y}_s^{(d)} - b_s^\alpha$. These need not be consecutive calendar weeks. A hat marks a raw scale estimate, and Φ is the standard normal distribution function. The default scales are

$$\widehat{\sigma}_{t,\text{med}} = \left[\frac{1}{|\mathcal{H}_t^{(d)}|}\sum_{s \in \mathcal{H}_t^{(d)}} (e_s^{.5})^2\right]^{1/2}, \tag{52}$$

$$\widehat{\sigma}_{t,\text{res}}^\alpha = \left[\frac{1}{|\mathcal{H}_t^{(d)}|}\sum_{s \in \mathcal{H}_t^{(d)}} (e_s^\alpha)^2\right]^{1/2}. \tag{53}$$

Both require five consumed rows. RMS includes bias because the errors are not centred. The optional robust alternative is median $|e_s^\alpha|/\Phi^{-1}(.75)$.

The common scale inferred from the current base is

$$\widehat{\sigma}_{t,\text{base}} = \frac{b_t^{.9} - b_t^{.1}}{\Phi^{-1}(.9) - \Phi^{-1}(.1)}. \tag{54}$$

The denominator is approximately 2.563. Base-width scales respond immediately to changes in the current forecast; residual scales respond only after the corresponding reports arrive.

Floor raw scales at 10^{-6}, then clip to $[.05S_0, S_0]$ once S_0 exists; denote these bounded scales by σ. The bounds limit how much the scale suppresses or amplifies accumulated corrections. They are implementation defaults, not requirements of studentization.

Table 1: Studentized methods. Scales refer to the bounded versions above. Initial states are specified before the first update.

Method ID	Centre and scale	Initialization and purpose
`studentized`	$a_t^\alpha = b_t^5$; common $\sigma_{t,\text{med}}$	$\tilde{u}_{\text{init}}^\alpha = \Phi^{-1}(\alpha)$. Reconstruct the distribution around the base median using recent errors.

Continued on next page

Table 1 (continued)

Method ID	Centre and scale	Initialization and purpose
`studentized_base`	$a_t^\alpha = b_t^5$; common $\sigma_{t,\text{base}}$	$\tilde{u}_{\text{init}}^\alpha = \Phi^{-1}(\alpha)$. Reconstruct the distribution from the median and the central width of the current base forecast.
`median_constant`	$a_t^\alpha = b_t^5$; common constant σ	$\tilde{u}_{\text{init}}^\alpha = \Phi^{-1}(\alpha)$; freeze $\sigma_{t,\text{med}}$ at the first reference week after both required warmups. Separates replacing the base by its median from subsequently changing the scale.
`studentized_zero`	$a_t^\alpha = b_t^5$; common $\sigma_{t,\text{med}}$	$\tilde{u}_{\text{init}} = 0$. Start all levels at the median; use the rate selected for `studentized`.
`median_constant_zero`	$a_t^\alpha = b_t^5$; common constant σ	$\tilde{u}_{\text{init}} = 0$; use the scale and rate of `median_constant`.
`studentized_full`	$a_t^\alpha = b_t^\alpha$; common $\sigma_{t,\text{base}}$	$\tilde{u}_{\text{init}} = 0$. Keep the full base and rescale its accumulated corrections together.
`studentized_level_rms`	$a_t^\alpha = b_t^\alpha$; $\sigma_{t,\text{res}}^\alpha$	$\tilde{u}_{\text{init}} = 0$. Set the scale at each level from recent errors at that level; weighted isotonic projection.

The scale of `median_constant` is fixed within each reconstructed history but can change between re-runs, unlike the reference scales (Section 3.3). Zero-start variants compare initialization at the same selected rate. Their first active forecast may already include a feedback update.

The residual-median methods also allow $t_5^{-1}(\alpha)$ initialization; the base-width median method always uses normal quantiles. It retains the median and bounded central width but imposes a symmetric normal shape in counts, so it need not reproduce the original forecast. Until a required residual scale exists, return the original full base. Feedback consumed while waiting is not replayed; later feedback on those rounds is compared with the original forecasts.

Studentization rescales the accumulated correction, rather than only the newest increment. Quantile-specific scales retain the coverage target, but no calibration guarantee is established here for those variants.

Relation to the `log1p` tracker. Standard MultiQT on the `log1p` tracker (`fixed_log`, `leaky_log`) nevertheless acts like a studentized method whose scale is the level. Its base in tracker units is $\beta_t^\alpha = \psi(b_t^\alpha) = \log(1 + b_t^\alpha)$. Without crossings, (32) gives $q_t^\alpha = \psi^{-1}(\beta_t^\alpha + \tilde{\theta}_t^\alpha)$, that is,

$$1 + q_t^\alpha = (1 + b_t^\alpha)\, e^{\tilde{\theta}_t^\alpha}, \qquad q_t^\alpha - b_t^\alpha = (1 + b_t^\alpha)(e^{\tilde{\theta}_t^\alpha} - 1) \approx (1 + b_t^\alpha)\, \tilde{\theta}_t^\alpha. \tag{55}$$

The state does not add counts to the base: it scales the shifted base $1 + b_t^\alpha$ by the factor $e^{\tilde{\theta}_t^\alpha}$ (the shift by one only keeps zero counts finite on the log scale). The correction in counts is therefore proportional to the current level.. The `identity` tracker, (31), adds the same number of counts at every level.

The state is the threshold for the residual on the log scale: level α is covered if $\psi(y) - \beta_t^\alpha \le \tilde{\theta}_t^\alpha$ (without crossings). This residual is measured relative to the level. Exactly,

$$\psi(y) - \beta_t^\alpha = \log\Big(1 + \frac{y - b_t^\alpha}{1 + b_t^\alpha}\Big) \approx \frac{y - b_t^\alpha}{1 + b_t^\alpha}, \tag{56}$$

with $\log(1 + x) \approx x$ for small relative errors x. A log-scale residual is thus approximately a count residual divided by the level: the studentized score (13) with $\sigma_t^\alpha = 1 + b_t^\alpha$, with $\tilde{\theta}_t^\alpha$ in the role of $\tilde{u}_t^\alpha$ in (14). The approximation only interprets the scale; the tracker uses the exact

coverage events and (32). Like $\sigma_{t,\text{base}}$, this scale is known at the cutoff, with no feedback lag, and a change of it rescales the whole accumulated correction (Section 2.1.5).

In general, a tracker transform studentizes by the reciprocal of its derivative at the base, since $\psi(y) - \beta_t^\alpha \approx \psi'(b_t^\alpha)(y - b_t^\alpha)$, and likewise for χ: $1/\psi'(b_t^\alpha) = 1 + b_t^\alpha$ for `log1p` and one for `identity` (the additive correction (31) of `fixed` and `leaky`); for the square-root trackers χ of `ref_op` and `sqrt_constant` (Section 3.5.4), $1/\chi'(b_t^\alpha)$ is a multiple of $\sqrt{b_t^\alpha}$, in between. Section 4.5.1 shows what level-tied scales do after peaks.

3.5.3 Weighted experts

An expert is a complete forecasting procedure; a bank is the set being combined. Weights learn from quantile losses separately for each location–horizon case. An adaptive rate changes their responsiveness.

3.5.3.1 Mixtures and feedback losses For K experts, let $f_{k,t}^\alpha$ be the forecast of expert k and w_k^α its weight. Suppressing time indices for one forecast, form the mixture $\hat{q}$ and repair any crossings:

$$\hat{q}^\alpha = \sum_{k=1}^{K} w_k^\alpha f_k^\alpha, \qquad w_k^\alpha \geq 0, \quad \sum_{k=1}^{K} w_k^\alpha = 1, \qquad q = \Pi_{\mathcal{K}}(\hat{q}). \tag{57}$$

Per-quantile weights can introduce crossings, repaired by isotonic projection. Common weights replace w_k^α by w_k and preserve order when every expert is ordered. All mixtures average counts.

Let n count processed feedback updates, with forecast row $s(n)$ and its report $\bar{y}_{s(n)}^{(d)}$ (29); in a re-run at reference week t, the report current then, $y_{s(n)}^{[\iota_t]}$, takes its place. Score the expert forecasts for that row, using L for unnormalized losses and ℓ for losses normalized by L_0:

$$L_{k,n}^\alpha = \rho_\alpha\left(f_{k,s(n)}^\alpha / c_{\text{scale}}, \bar{y}_{s(n)}^{(d)} / c_{\text{scale}}\right), \qquad L_{k,n} = \frac{1}{m}\sum_{\alpha\in\mathcal{A}} L_{k,n}^\alpha, \tag{58}$$

$$\ell_{k,n}^\alpha = L_{k,n}^\alpha / L_0, \qquad \ell_{k,n} = \frac{1}{m}\sum_{\alpha\in\mathcal{A}} \ell_{k,n}^\alpha. \tag{59}$$

The same L_0 applies to all experts and levels within a case and feedback design (Section 3.5.3.4). Score the originally computed forecasts in stateful learning and the reconstructed forecasts in a re-run.

3.5.3.2 Exponential updates and optional sharing Let $w_{k,n}^\alpha$ and λ_n^α be the weight and rate before update n. Before optional sharing, update separately at each level to obtain $\bar{w}_{k,n+1}^\alpha$:

$$\bar{w}_{k,n+1}^\alpha = \frac{w_{k,n}^\alpha \exp(-\lambda_n^\alpha \ell_{k,n}^\alpha)}{\sum_{j=1}^{K} w_{j,n}^\alpha \exp(-\lambda_n^\alpha \ell_{j,n}^\alpha)}. \tag{60}$$

With common weights, there is one update using the mean loss:

$$\bar{w}_{k,n+1} = \frac{w_{k,n} \exp(-\lambda_n \ell_{k,n})}{\sum_{j=1}^{K} w_{j,n} \exp(-\lambda_n \ell_{j,n})}. \tag{61}$$

Lower loss increases relative weight; larger λ strengthens the response. Weights start uniformly, and $\lambda = 0$ keeps equal weights.

When sharing is enabled with fraction γ, follow the exponential step by

$$w_{k,n+1}^\alpha = (1-\gamma)\bar{w}_{k,n+1}^\alpha + \gamma/K. \tag{62}$$

Omit α for common weights. Sharing helps downweighted experts recover: use $\gamma = 1/(2H)$ when enabled and zero otherwise. Table 2 distinguishes sharing from common weights across levels.

3.5.3.3 Adaptive learning rates The lookback H counts processed updates. The first H updates use starting rate λ_{start}. For rate-grid and weekly-start mixtures, record the weighted mean squared loss of the experts, using weights before the exponential step and sharing:

$$M_n^\alpha = \sum_{k=1}^{K} w_{k,n}^\alpha (\ell_{k,n}^\alpha)^2,$$
$$M_n = \sum_{k=1}^{K} w_{k,n} \ell_{k,n}^2. \tag{63}$$

The score mixture `score_learned` instead uses the weighted variance (`centered=TRUE`):

$$M_n^\alpha = \sum_{k=1}^{K} w_{k,n}^\alpha \left(\ell_{k,n}^\alpha - \sum_{j=1}^{K} w_{j,n}^\alpha \ell_{j,n}^\alpha \right)^2. \tag{64}$$

For this centred rule, treat variances at most $10^{-12} \sum_k w_{k,n}^\alpha (\ell_{k,n}^\alpha)^2$ as zero, and analogously for common weights, to avoid large rates from numerical rounding. For per-quantile weighting, the rate is

$$\lambda_n^\alpha = \begin{cases} \lambda_{\text{start}}, & 1 \le n \le H, \\ \sqrt{\dfrac{\log(2KH)+1}{\sum_{r=n-H}^{n-1} M_r^\alpha}}, & n > H. \end{cases} \tag{65}$$

Only the previous H values of M_r^α enter; a zero sum retains the previous finite rate. Larger values reduce the rate.

Why the score mixture uses the variance. The step (60) is unchanged when the same amount is added to every expert's loss: the factor $e^{-\lambda c}$ cancels in the normalization. Only the differences between experts move the weights. The weighted mean squared loss (63), however, also grows with a loss that all experts share, and so lowers the rate (65) without any information for the weights. Since the step ignores a common shift, the losses can be centred at their weighted mean before squaring; M_n^α is then the weighted variance (64), which sets the rate by the differences alone. The score members are complete corrections of the same base, so their losses are dominated by the base error they share: in the exploratory check of the score bank the weights under the weighted mean squared loss hardly left uniform, which motivated the centred rule. In the saved runs for COVID, $h = 2$, delay 0 (the five plotted locations), the between-expert variance under equal weights is 2% of the weighted mean squared loss for the score bank, so the centred rate is about seven times larger. The same holds for the weekly starts (2%) and, less so, for the rate grid (18%); these mixtures keep the weighted mean squared loss of Gibbs and Candès [3], whose regret bound (their Lemma 2) gives the constant $\log(2KH)+1$ for $\gamma = 1/(2H)$.

Per-quantile weighting maintains a vector, a history of M_r^α and a rate for each level. Common weighting uses one of each: average losses across levels *before* computing M_r, then omit α in (65).

Later starts and implementation. For a released row s, only the weekly-start copies started at or before s are eligible: they issued a forecast for that row. Normalize their weights for the step (60), then restore their total mass; sharing (62) spreads γ times that mass over the eligible copies only. Copies started later keep their weights. Use the eligible count as K in (63)–(65). Updates with fewer than two eligible experts do not count toward H. At each reference week, first process the released feedback, then admit the new copy with $1/K$ of the enlarged bank, preserving the existing relative weights, then issue the mixture. For `restart_recent`, a copy that would make the bank larger than K_{recent} retires the oldest one first: its weight goes to the remaining copies in proportion to their weights, and it is never eligible again.

3.5.3.4 Expert loss normalization Using the quantile loss in (1), define one reference loss per case and delay:

$$L_0 = \max\left\{\varepsilon_L, \frac{1}{20|\mathcal{A}|}\sum_{s\in I_{20}^{(d)}}\sum_{\alpha\in\mathcal{A}}\rho_\alpha\left(\frac{b_s^\alpha}{c_{\text{scale}}}, \frac{\bar{y}_s^{(d)}}{c_{\text{scale}}}\right)\right\}, \qquad \varepsilon_L = 10^{-6}. \tag{66}$$

Losses are relative to the initial mean base loss.

Initial and fixed-rate weight updates. Dividing by L_0 preserves rankings and makes weight changes depend on relative loss gaps. A change of count units scales both L and L_0 equally, leaving weights unchanged when the numerical floor is inactive. This matters throughout fixed-rate runs and during the first H adaptive-run updates at λ_{start}; these are distinct from the 20 rows used to estimate L_0.

Effect on adaptive learning rates. After H updates, the rule in (65) already adapts to the losses through M_r^α. Use superscripts raw, norm for calculations without and with L_0 normalization. Conditional on the same pre-update weight vectors at updates $n-H,\dots,n$ and a positive sum of M_r^α, normalization gives, separately at each level,

$$\begin{aligned} M_r^{\alpha,\text{norm}} &= M_r^{\alpha,\text{raw}}/L_0^2, & \lambda_n^{\alpha,\text{norm}} &= L_0\lambda_n^{\alpha,\text{raw}}, \\ \lambda_n^{\alpha,\text{norm}}\ell_{k,n}^\alpha &= \lambda_n^{\alpha,\text{raw}}L_{k,n}^\alpha. \end{aligned} \tag{67}$$

The identity holds for both definitions of M_r^α and for common weights. The numerical zero-variance threshold of the centred rule scales by L_0^{-2} too. Thus the loss and rate changes cancel in the exponential update, conditional on the same weight history. Runs with the same numerical starting rate can still differ: their first H updates change later weights and M_r^α.

Equivalent runs require matching settings and $\lambda_{\text{start}}^{\text{raw}} = \lambda_{\text{start}}^{\text{norm}}/L_0$, including the same scaling of retained rates when the sum of M_r^α is zero. Keep normalization throughout a run so incoming losses, stored M_r^α and rates remain in consistent units. After adaptation, L_0 is not an additional reduction in update strength.

3.5.3.5 Expert combinations implemented in the code Table 2 groups the current combinations. Every learned mixture uses the adaptive rate above, with the weighted mean squared loss for rate-grid and weekly-start mixtures and the weighted variance for the score mixture.

Method IDs	Weighting and bank
`uniform`	Equal weights on 12 persistent rate experts.
`weighted`, `shared`	Same rate bank; per-quantile weights, without/with sharing.
`weighted_common`, `shared_common`	Same rate bank; common weights, without/with sharing.
`restart_uniform`	Equal weights on all weekly starts so far, rate $.05S_0$.
`restart_weighted`	Same starts; adaptive λ, sharing, per-quantile weights.
`restart_recent`	The eight most recent starts; adaptive λ, sharing, per-quantile weights; the oldest retires when a new copy starts.
`restart_recent_dual`	Two copies per start, rates $.03S_0$ and $.3S_0$; the 16 copies of the eight most recent starts; adaptive λ, sharing, per-quantile weights.
`score_equal`	Equal weights on the declared score bank.
`score_learned`	Same score bank; adaptive λ, sharing, per-quantile weights.

Table 2: Current expert combinations.

The four `_outer` variants of `shared`, `shared_common`, `restart_uniform`, and `restart_weighted` add persistent MultiQT at $.001S_0$ to their inner mixture, retaining its settings and learning from their own final forecasts. Rate selection and fixed $H = 8$ are described in Section 3.3.

Rate and weekly-start banks. The rate bank uses

$$\kappa \in \{0, .01, .015, .02, .03, .05, .075, .1, .15, .2, .3, .45\};$$

all experts start together and the zero-rate member is the original. The weekly-start bank adds a fresh copy of Fixed MultiQT, with rate $.05S_0$, at every retained reference week once initialization permits; with warmup replay, starts begin in the first round. The copy started at week t_k issues the base at t_k, with zero correction, and then learns only from the reports on its own forecasts, the rows $s \geq t_k$; reports on earlier rows are ignored. Older copies keep their accumulated corrections, and there is no permanent zero-rate expert. At week t the bank thus holds corrections learned over every window that ends at t and starts at a retained week; the newest copy is still the base. Starts are counted in retained reference weeks, not elapsed weeks, so a gap without forecasts adds no copies.

The three mixtures weigh this bank differently. `restart_uniform` gives all copies started so far equal weights and learns nothing; `restart_weighted` learns their weights as in (60)–(65), always with sharing $\gamma = 1/(2H)$ (unlike the rate grid, there is no weekly-start mixture without sharing). `restart_recent` learns the weights of the $K_{\text{recent}} = 8$ most recent copies $\mathcal{B}_t$ in the same way, also with sharing, or of all copies if fewer have started; older copies retire:

$$\hat{q}_t^{\alpha} = \sum_{k \in \mathcal{B}_t} w_{k,t}^{\alpha} f_{k,t}^{\alpha}, \qquad \sum_{k \in \mathcal{B}_t} w_{k,t}^{\alpha} = 1, \qquad q_t = \Pi_{\mathcal{K}}(\hat{q}_t). \tag{68}$$

This limits the age of the corrections without detecting peaks: no correction older than eight starts enters the forecast. A new copy receives weight after the released feedback is processed and before the forecast is issued, so a new color band in the weight plots alone does not show lower loss.

`restart_recent_dual` starts two copies at every retained week, a slow one with rate $.03S_0$ and a fast one with $.3S_0$. It learns the weights of the 16 copies of the eight most recent starts as in (68), with sharing; both copies of the oldest start retire together. Within the same recent window, the weights can thus choose between a slow and a fast correction.

The five score-based experts and fixed membership. The bank compares ways to retain the base or correct its errors. Write $S_{k,t}^{\alpha}(y)$ for the residual score of member k. The members are the base, the two additive trackers, the best studentized method and Delphi's square-root tracker; the base-width studentized methods are left out (Section 4.5.1). All members except `sqrt_constant` track counts; for their scores, $r = y$, projected as in (50). Fixed, leaky and studentized experts use separately selected rates, `sqrt_constant` Delphi's constant rate, and the original does not update. This compares complete procedures, not scores at a common rate.

Membership is fixed across reference weeks, horizons and re-runs, even when forecasts coincide. Do not deduplicate complete forecast histories: later forecasts or revisions must not change earlier membership.

Table 3: The declared score bank. Scales are the bounded scales defined earlier.

Member	Score or forecast	Purpose
`original`	$f^{\alpha}_{k,t} = b^{\alpha}_t$	Preserve the base as an option.
`fixed`	$S^{\alpha}_{k,t}(y) = r - b^{\alpha}_t$	Persistent additive correction; $\eta = \kappa S_0$.
`leaky`	Fixed MultiQT with leak $\delta = .1$, (49)	Additive correction that fades back to the base unless feedback renews it.
`studentized_level_rms`	$S^{\alpha}_{k,t}(y) = (r - b^{\alpha}_t)/\sigma^{\alpha}_{t,\mathrm{res}}$	Keep the full base; scale each level's correction by its recent errors.
`sqrt_constant`	$\chi(y) - \chi(b^{\alpha}_t)$, (73)	Correction on the square-root scale, between additive and level-proportional.

Initialization and scales follow Table 1. Leaky changes the state update rather than defining a new static residual score.

Equal and learned score mixtures. `score_equal` averages the declared experts. For `score_learned`, learn one vector per quantile level using the normalized losses, adaptive rate and sharing above, then repair crossings by isotonic projection.

3.5.4 Reference benchmarks

The references are the original forecast, a hindsight control and two production configurations of Delphi [4].

Original forecast: `original`. Return $q_t = b_t$ without recalibration or learning. This is the reference for the reported loss changes and reductions.

Target oracle: `oracle_target`. This hindsight control returns

$$q^{\mathrm{oracle}}_t = \begin{cases} q^{\mathrm{fixed}}_t, & T_t \le p(t), \\ b_t, & T_t > p(t). \end{cases} \tag{69}$$

This bypasses correction for postpeak targets, including forecasts made before the peak. Fixed MultiQT still learns from its own forecasts. The control tests bypassing correction with perfect phase information; it is not deployable. Setting the rate to zero would instead freeze the offset, giving a different control.

Delphi benchmarks: separate scales and initialization. Both run for both diseases and all feedback designs, independently of the identity/log setting. States start at zero without S_0 or warmup; constants are untuned. For transform χ and projected forecast ξ_t in those units, the released coverage-error sum is

$$\epsilon^{\alpha}_t = \sum_{s \in \mathcal{R}^{(d)}_t} \left(\alpha - \mathbf{1}\{\chi(\bar{y}^{(d)}_s) \le \xi^{\alpha}_s\} \right). \tag{70}$$

In a weekly re-run, use its current reports and reconstructed forecasts, as in Section 3.2.

Adaptive fast and slow offsets: `ref_op`. Let $\mathcal{F}_t^{(d)} = \bigcup_{j \le t} \mathcal{R}_j^{(d)}$ contain all consumed rows, and $\mathcal{F}_{t,20}^{(d)}$ the last 20 in release order, or fewer if unavailable. The COVID production configuration uses $\chi(x) = \sqrt{x}$ on raw counts, with rates

$$\eta_t^F = \max\left\{10^{-3},\ .03Q_{.9}\left(\left\{|\sqrt{\bar{y}_s^{(d)}} - \sqrt{b_s^\alpha}| : s \in \mathcal{F}_{t,20}^{(d)},\ \alpha \in \mathcal{A}\right\}\right)\right\},$$
$$\eta_t^S = .003Q_{.9}\left(\left\{|\sqrt{\bar{y}_s^{(d)}} - \sqrt{b_s^\alpha}| : s \in \mathcal{F}_t^{(d)},\ \alpha \in \mathcal{A}\right\}\right). \tag{71}$$

Both rates are zero before the first release, pool absolute base errors over levels, and are recomputed as feedback arrives. The fast and slow hidden states $\tilde{\theta}_t^F, \tilde{\theta}_t^S$ and forecast are

$$\tilde{\theta}_t^{F,\alpha} = .9\tilde{\theta}_{t-1}^{F,\alpha} + \eta_t^F \epsilon_t^\alpha, \qquad \tilde{\theta}_t^{S,\alpha} = \tilde{\theta}_{t-1}^{S,\alpha} + \eta_t^S \epsilon_t^\alpha,$$
$$\xi_t = \Pi_{\mathcal{K}}(\sqrt{b_t} + \tilde{\theta}_t^F + \tilde{\theta}_t^S), \qquad q_t^\alpha = (\max\{\xi_t^\alpha, 0\})^2. \tag{72}$$

The fast correction loses 10% per processed reference week, even without feedback; gaps do not compound decay. The slow correction persists. Re-runs reconstruct both rates and states. These rolling scales are separate from the frozen S_0.

Constant square-root rate: `sqrt_constant`. The flu production benchmark uses $\chi(x) = \sqrt{x/c_{\text{pop}}}$, where c_{pop} always equals population/100,000. Its update and forecast are

$$\tilde{\theta}_t^\alpha = \tilde{\theta}_{t-1}^\alpha + .018\epsilon_t^\alpha,$$
$$\xi_t = \Pi_{\mathcal{K}}\left(\sqrt{b_t/c_{\text{pop}}} + \tilde{\theta}_t\right), \qquad q_t^\alpha = c_{\text{pop}}(\max\{\xi_t^\alpha, 0\})^2. \tag{73}$$

The rate .018 is in square-root counts-per-100,000 units. There is no adaptive rate, decay or second state. Delphi selected this constant using flu seasons 2024/25 and 2025/26, which overlap our evaluation period. Although we do not retune it, this is not an independently selected held-out benchmark for those seasons.

Zero boundary and plots. Square-root methods return zero when $\xi_t^\alpha < 0$ but learn from untruncated ξ_s^α. At zero outcomes, learning coverage can differ from coverage of the returned forecast. Both appear under "Single trackers" in script 04 and in the evaluation tree of script 06.

4 Results

All results come from the rerun of 11–12 November 2026 with the methods of Section 3.5: CMU–TimeSeries submissions as the base, tuning through 31 May 2025, evaluation from June 2025 through September 2026, scored against the latest counts. Unless stated otherwise, a number is the change of a pooled mean relative to the base, in percent: of the quantile loss per 100,000 ("loss") or of the calibration error (CE). Pooled means weight every location–horizon case equally over the 52 locations without the US and all horizons (Section 3.4); negative is better. "Up to peak" and "after peak" are the windows of Section 3.4. Each feedback design uses the rates selected with it. Where a table gives one number per method and window, it is the mean over the six disease–design cells.

The tables summarize; the plots in `output/pdf/` show every disease, horizon, design and plotted location. The pages cited below:

- `multiqt_phase_rate_tradeoffs.pdf` (script 02, Fixed MultiQT on the rate grid): pp. 1–3 pooled, one page per design (delay 0, delay 1, re-run), horizons −1 to 3 in the columns, loss and calibration error of each disease in the rows; pp. 4–6 the US.

- `multiqt_error_tradeoff.pdf` (script 02): p. 1 the pooled means of every disease, design and horizon; pp. 2–7 the single locations, COVID and then flu, one page per design.
- `multiqt_tuning.pdf` (script 05): the elbow curves, pp. 1–3 COVID and 4–6 flu, one page per design.
- `multiqt_peak_reasoning_tradeoff.pdf` (script 04, all methods): per disease the 52 locations (pp. 1 and 8), their means only (pp. 2 and 9) and the US, California, Texas, New York and Pennsylvania (pp. 3–7 and 10–14).
- `multiqt_feedback_bars.pdf` (every method, the three designs side by side) and `multiqt_evaluation_tree.pdf` (the ten best per page), script 06: per disease the pooled page (pp. 1 and 7) and the five locations (pp. 2–6 and 8–12). The method groups of the bar plot are those of the forecast pages.
- `multiqt_method_forecasts/` (script 04), one file per disease, horizon and design: seven pages per location, one per method group ("Single trackers", "Log1p tracker", "Rate-grid experts", "Weekly starts: immediate admission", "Score-based experts", "Studentized median", "Studentized full base"); US pp. 1–7, California 8–14, Texas 15–21, New York 22–28, Pennsylvania 29–35.
- `multiqt_weight_timelines/` (script 04), one file per disease, horizon and design: four pages per location in the same order: rate-grid weights (pp. 1, 5, 9, 13, 17), score experts (2, 6, 10, 14, 18), weekly starts (3, 7, 11, 15, 19) and two-rate weekly starts (4, 8, 12, 16, 20).

The bar plot and the tree show the loss reduction, $100(1 - \text{pooled loss/pooled base loss})$, positive is better, with each case weighted by its weeks; the tables give the change of the mean over cases, negative is better. Overall the two agree up to the sign within 0.5 points; in the peak windows, where cases have different numbers of weeks, they mostly differ by less than 0.1 points, at most by 9 (the median rebuilds up to peaks). The score tables are in `output/results/peak_reasoning/`. Each subsection ends with the plots that show its results.

4.1 Summary

The answers in brief, as means over the six disease–design cells of the change versus the base. The brackets name the table and the plots that show each result; "not plotted" marks numbers computed only for these notes.

- **Fixed rates.** Pooled over horizons, no rate beats the base overall in any design, and the loss grows with the rate. Calibration error falls quickly and levels off by $\kappa \approx .01$–$.03$. Only the rise to the peak gains, and longer horizons pay more. The exception is the COVID nowcast ($h = -1$), where $\kappa = .01$ beats the base. [Table 4; `multiqt_phase_rate_tradeoffs.pdf`, pp. 1–3, the horizons in the columns, $h = -1$ first; `multiqt_error_tradeoff.pdf`, p. 1.]
- **Feedback designs.** Delay 0 learns from first reports, which are about 7% too low (72–74% of them below the latest count; not plotted). At equal rates it therefore improves calibration much less than delay 1 or the re-run (COVID at $\kappa = .01$: -6% against -40% and -54%). With the selected rates, the comparison also reflects the different rates chosen per design. [Section 4.3. Equal rates: `multiqt_phase_rate_tradeoffs.pdf`, pp. 1–3, and `multiqt_error_tradeoff.pdf`, p. 1. Selected rates: Table 5, `multiqt_tuning.pdf`, `multiqt_feedback_bars.pdf`, pp. 1 and 7.]
- **Single trackers.** Only Delphi's square-root trackers stay near the base (+0.5%, +0.8%). Fixed MultiQT costs +16% and Leaky +26%, mainly through its larger selected rates. The `log1p` trackers are better up to peaks and on the log scale (log loss +11% against +51% for Fixed) but lose after peaks on counts (+65% and +59%). [Tables 6 and 5;

`multiqt_feedback_bars.pdf`, pp. 1 and 7, groups "Single trackers" and "Log1p tracker"; `multiqt_peak_reasoning_tradeoff.pdf`, pp. 2 and 9.]

- **Studentized methods.** None beats the base; the best, `studentized_level_rms`, costs +14.8%. The base-width scale fails after peaks (`studentized_base`, COVID +7% to +77%): the base width keeps growing after the peak, and the correction learned during the rise grows with it. [Table 7; Section 4.5.1, whose three facts are not plotted; `multiqt_feedback_bars.pdf`, groups "Studentized median" and "Studentized full base".]
- **CMU forecast versus its median.** Keeping all CMU quantiles beats rebuilding from the median by 22–32 points. Most of the value is the weekly width; the quantile shape adds about 5 points for flu. [Table 8; in `multiqt_feedback_bars.pdf`, Fixed MultiQT and the groups "Studentized median" and "Studentized full base".]
- **Rate grid.** Learned weights beat equal weights; one weight vector for all levels without sharing is best (+4.4%), and sharing costs about 4 points. [Table 9; `multiqt_feedback_bars.pdf`, group "Rate-grid experts".] The weights pick the rate per location: .03–.05 in California, Texas and the US, 0 in New York and Pennsylvania; the new rate .45 gets little weight. [Rate-grid weight pages, column "Learned, no sharing"; the mean weights quoted are not plotted.]
- **Weekly starts.** `restart_recent` is the best mixture (+1.9%), but its weights stay almost equal, so its gain comes from the eight-week window, not from learning. The two-rate version splits its weight about evenly between slow and fast copies: more loss (+10.3%), but the best after-peak calibration. [Table 9; `multiqt_feedback_bars.pdf`, group "Weekly starts: immediate admission"; weekly-start weight pages, column "Most recent starts: learned + sharing", and the two-rate weight pages.]
- **Score bank.** Equal and learned weights perform alike, and neither beats its member `sqrt_constant`. [Tables 9 and 6; `multiqt_feedback_bars.pdf`, groups "Score-based experts" and "Single trackers"; score-expert weight pages.]
- **Locations.** For COVID, California and Texas gain, and several methods beat the base there; New York, Pennsylvania and the US lose with most methods. For flu the after-peak loss decides. Calibration improves strongly for flu everywhere, but little for COVID in the large states. [Table 12; `multiqt_feedback_bars.pdf` and `multiqt_evaluation_tree.pdf`, pp. 2–6 (COVID) and 8–12 (flu), the coverage-error panels for calibration; `multiqt_peak_reasoning_tradeoff.pdf`, pp. 3–7 and 10–14.]

4.2 Fixed rates: what the trade-off plots show

Script 02 runs Fixed MultiQT, untuned, on a dense grid of rates $\eta = \kappa S_0$, $\kappa \in [.01, .45]$, for every location, horizon and feedback design. `multiqt_phase_rate_tradeoffs.pdf` shows the pooled loss and calibration error against κ by horizon, overall, up to and after peaks; `multiqt_error_tradeoff.pdf` plots calibration error against loss for each location and horizon. Table 4 averages the pooled changes over the three feedback designs.

		κ				
Disease	Change vs base (%)	.01	.03	.1	.3	.45
COVID	Loss, overall	1.5	6.0	27.7	99.5	152.9
	Loss, up to peak	−3.0	−3.5	19.0	96.5	139.6
	Loss, after peak	4.0	10.9	22.2	43.5	62.8
	CE, overall	−32.9	−44.6	−52.7	−59.7	−63.1
Flu	Loss, overall	1.5	6.8	27.0	87.8	135.1
	Loss, up to peak	−1.8	−1.4	0.4	10.4	20.2
	Loss, after peak	0.4	2.7	15.0	56.4	83.2
	CE, overall	−38.6	−40.3	−43.5	−48.1	−51.4
	CE, after peak	−0.2	1.0	7.8	11.7	9.4

Table 4: Fixed MultiQT on the rate grid (script 02), mean over the three feedback designs of the pooled change versus the base.

- **No rate beats the base overall.** In every disease and design the pooled overall loss is lowest at $\kappa = 0$ and grows with the rate: by about 1.5% at $\kappa = .01$, 6–7% at .03 and 90–100% at .3.
- **Calibration improves fast, then saturates.** Most of the reduction in calibration error is reached by $\kappa = .01$–.03, while the loss keeps rising. Where calibration is the aim, small rates are the efficient end of the trade-off.
- **Only the rise gains.** Up to peaks, small rates lower the loss (COVID about −3%, flu about −2%); after peaks every rate raises it. For flu, the after-peak calibration error also grows with the rate: corrections learned during the rise persist into the decline.
- **Longer horizons pay more.** At $\kappa = .1$ the COVID loss rises by 8% at $h = -1$ and by 42% at $h = 3$ (flu: 9% and 31%), for similar calibration gains. At the nowcast horizon, $\kappa = .01$ even lowers the COVID loss (−2.6%) and its calibration error by 54%: the only horizon where Fixed MultiQT beats the base overall.
- **Locations** (`multiqt_error_tradeoff.pdf`) follow the same pattern: larger rates move the points towards lower calibration error and higher loss, separately up to and after peaks.

Plots. The curves behind Table 4: `multiqt_phase_rate_tradeoffs.pdf`, pp. 1–3 (pooled, one page per feedback design: loss and calibration error against κ by horizon and window; the US on pp. 4–6). The trade-off itself: `multiqt_error_tradeoff.pdf`, p. 1 (pooled means only, every disease, design and horizon) and pp. 2–7 (single locations).

4.3 Feedback designs

The base forecasts and the scoring target are the same in every design; only the reports the trackers learn from differ (Section 3.2). Two comparisons separate report timing from the selected rates.

At equal rates (script 02), delay 1 and the weekly re-run reduce calibration error far more than delay 0: at $\kappa = .01$, COVID −40% and −54% against −6%, flu −42% and −41% against −33%. Up to peaks they also lower the loss more (COVID at $\kappa = .02$: −5% and −7% against −1%); after peaks they cost a little more (COVID at $\kappa = .01$: +4% and +5% against +2%). The reason is the first report. Delay 0 learns from it, and it is too low: its median is 0.93 of the latest count for both diseases, 72% (COVID) and 74% (flu) of the rows lie below the latest

count, and 56% and 57% more than 5% below. One week later the median is 1.00 and about half of the rows lie below. A tracker that learns from first reports thus learns to forecast too low for the latest counts it is scored against. These report ratios are computed from the prepared reports; no plot shows them yet.

With the selected rates (script 05, Table 5), the designs also differ in the rate the elbow rule picks: for COVID, Fixed MultiQT gets $\kappa = .01$ at delay 0 but .1 at delay 1 and in the re-run. The tuned methods at delay 1 and in the re-run therefore trade much more loss for calibration (Section 4.4); differences between designs for tuned methods mix report timing and the selected rate.

	COVID			Flu		
Method	delay 0	delay 1	re-run	delay 0	delay 1	re-run
`fixed`	.01	.1	.1	.05	.05	.05
`leaky`	.1	.1	.1	.2	.1	.2
`fixed_log`	.01	.2	.2	.2	.1	.2
`leaky_log`	.05	.2	.2	.3	.2	.3
`studentized_level_rms`	.03	.3	.3	.3	.1	.3
`studentized_full`	.1	1*	.3	.3	.3	.3
`studentized_base`	.1	1*	.3	.3	.3	.3

Table 5: Selected rates (κ; for the studentized methods η in units of σ_t). *: upper edge of the grid.

Leaky's grid was extended from .2 to 1 after its selected rate sat at that upper edge in seven of twelve cells. With the wider grid only the `log1p` rate for flu at delay 0 and in the re-run moves (to .3); the others were genuine elbows. With the leak, corrections fade, so the rule selects larger steps than for Fixed MultiQT.

Plots. At equal rates: `multiqt_phase_rate_tradeoffs.pdf`, pp. 1–3, and the rows of `multiqt_error_tradeoff.pdf`, p. 1. With the selected rates: `multiqt_feedback_bars.pdf` (every method, the three designs side by side, with the tree's values and axes; pooled on pp. 1 and 7) and the first split of `multiqt_evaluation_tree.pdf` (pp. 1 and 7). The selected rates and their elbow curves: `multiqt_tuning.pdf`.

4.4 Single trackers: additive, leaky, log1p and Delphi's

Table 6 compares the single trackers; Table 5 lists their selected rates.

	Loss			Log loss	CE	Cases better (%)	
Method	overall	up to peak	after peak	overall	overall	COVID	flu
`fixed`	16.2	5.4	11.2	51.1	−41.9	14	4
`leaky`	26.2	9.5	15.2	79.4	−41.1	11	2
`fixed_log`	31.5	3.3	65.0	10.5	−42.1	21	5
`leaky_log`	27.5	2.6	58.7	10.7	−36.1	24	5
`ref_op`	0.5	−1.7	2.8	0.3	−20.3	49	32
`sqrt_constant`	0.8	−3.6	4.1	−0.8	−30.2	32	37
`oracle_target`	2.7	5.4	0.0	11.2	−14.9	33	39

Table 6: Single trackers: pooled change versus the base (%), mean over the six disease–design cells. Cases better: share of location–horizon–design cases with lower overall loss than the base.

- **Only Delphi's square-root trackers stay near the base.** `ref_op` and `sqrt_constant` change the loss by +0.5% and +0.8% and reduce calibration error by 20% and 30%. They gain up to peaks (−2%, −4%), lose a little after them (+3%, +4%) and beat the base in a third to a half of the cases, untuned. Their correction scales with $\sqrt{b}$, between additive and level-proportional (Section 3.5.2).
- **Fixed MultiQT** trades +16% loss for −42% calibration error, the elbow rule's choice; for COVID at delay 0, where $\kappa = .01$ is selected, it costs only +2% (Section 4.3).
- **Delphi's leak** costs more loss than Fixed (+26%), mainly through its larger selected rates (Table 5). Where the leak should help, after COVID peaks at delay 1 and in the re-run, it loses less than Fixed (+13% and +12% against +24%).
- **The log1p trackers** are better than the additive ones up to peaks (`fixed_log` +3% against +5%; flu at delay 1 −8%) and on the log scale, their own criterion (log loss +11% for both, against +51% and +79%). On counts, they lose after peaks (+65% and +59%; `fixed_log` for flu at delay 0 and in the re-run +103% and +126% against +6% for Fixed). This is the level scaling of Section 3.5.2: the correction is issued in proportion to the base level, which stays high after the peak while the counts fall (Section 4.5.1).
- **Knowing the peak does not rescue Fixed.** The hindsight `oracle_target`, Fixed up to each peak and the base after it, still loses 2.7%: with the selected rates, Fixed's corrections already cost loss up to peaks.

Plots. `multiqt_feedback_bars.pdf`, groups "Single trackers" and "Log1p tracker" (pooled: pp. 1 and 7); `multiqt_peak_reasoning_tradeoff.pdf`, pp. 2 and 9 (means over the 52 locations by design, window and horizon); the forecasts on the pages "Single trackers" and "Log1p tracker" of `multiqt_method_forecasts/` (the first two of each location's seven).

4.5 Studentized methods

	Loss			Log loss	CE	
Method	overall	up to peak	after peak	overall	overall	after peak
`studentized_level_rms`	14.8	1.2	24.5	21.1	−43.0	17.3
`studentized_full`	20.8	7.6	33.5	30.1	−42.0	2.5
`studentized_base`	23.0	9.7	37.4	30.2	−42.6	4.1
`studentized`	36.7	25.1	49.2	38.4	−36.4	26.4
`studentized_zero`	39.8	29.7	53.5	39.8	−20.2	36.9
`median_constant`	47.8	23.6	31.2	82.1	−42.2	15.1
`median_constant_zero`	49.0	29.6	38.2	81.9	−37.9	30.3
`fixed` (reference)	16.2	5.4	11.2	51.1	−41.9	−1.0

Table 7: Studentized methods: pooled change versus the base (%), mean over the six disease–design cells.

No studentized method beats the base in pooled loss. The best, `studentized_level_rms` (full base, each level scaled by its recent errors), is slightly better than Fixed overall (+14.8% against +16.2%) and up to peaks (+1.2% against +5.4%), and close to the base for COVID at delay 0 (+0.8%); after peaks it loses more (+24.5%) and its calibration error grows (+17%), because its residual scale lags the turn. The two base-width methods lose 21% and 23%, almost all after peaks (Section 4.5.1). The methods that rebuild the forecast from the median lose 37% to 49% (Section 4.6). The `log1p` trackers, studentized by the level, fail after peaks in the same way (Section 4.4).

Plots. `multiqt_feedback_bars.pdf`, groups "Studentized median" and "Studentized full base" (pooled: pp. 1 and 7); `multiqt_peak_reasoning_tradeoff.pdf`, pp. 2 and 9; the forecasts on the pages "Studentized median" and "Studentized full base" of `multiqt_method_forecasts/` (the last two of each location's seven).

4.5.1 Why the base-width scale fails after peaks

`studentized_base` uses the base-width scale (54). It reacts to the current base without feedback delay: it is large when the base is uncertain and small otherwise. It nevertheless raises the pooled loss in every disease and feedback design (overall +4% to +58%), while it reduces the calibration error about as much as `fixed`. Up to peaks it is close to the base except for COVID at delay 1, where its selected rate is at the upper edge of the grid; after peaks it loses 29–39% for flu and 7–77% for COVID (7% at delay 0), against 2–24% for `fixed`.

Three facts explain this (medians over locations, peaks and horizons 0–3, relative to the peak week):

- The base is too low up to the peak: its median is 0.4–0.85 of the latest counts, and 18–74% of the counts lie above its 0.9 quantile. The tracker therefore accumulates an upward correction, $\tilde{u}_t > 0$.
- The base width does not turn with the counts. Two weeks after the peak the counts have fallen to 0.72 (COVID) and 0.56 (flu) of their peak value, while the width has grown to 1.13 and 1.34 of its peak-week value; four weeks before the peak it was 0.76 and 0.57.
- The forecast $b_t + \sigma_t \tilde{u}_t$ rescales the whole accumulated correction with the current scale, not only the newest step (Section 2.1.5). The upward correction learned during the rise is therefore issued 1.5 (COVID) to 2.4 (flu) times larger two weeks after the peak, when the base is already high enough (flu: median 1.07 of the counts) or too high (COVID: 1.25).

`fixed` keeps its correction constant in counts: it carries the same leftover correction past the peak, but does not enlarge it. The fast rates selected for the studentized methods (0.1–1 in units of σ_t) reinforce the effect. The scale measures the base's own uncertainty, and that lags the turn of the counts.

Standard MultiQT on the `log1p` tracker rescales its state in the same way, with the level $1+b_t^\alpha$ in place of σ_t, (55); two weeks after the peak the base median is 1.27 (COVID) and 1.38 (flu) of its peak-week value. The results confirm it: `fixed_log` and `leaky_log` lose 65% and 59% after peaks. The square-root trackers scale by about the square root of these ratios and stay within 1% of the base's loss on average. The three facts are computed from the prepared forecasts and counts; no plot shows the base width around peaks yet (`multiqt_base_forecast_paths.pdf` shows the base medians).

4.6 Does the CMU forecast offer more than its median?

Three pairs of methods use the same scale and rate tuning, one correcting all CMU quantiles and one rebuilding the forecast from the base median with normal quantiles:

Scale: full base vs median rebuild	Loss, overall		Full minus median (points)		
	full	median	overall	up to peak	after peak
Constant: `fixed` vs `median_constant`	16.2	47.8	−31.6	−18.1	−20.0
Recent errors: `studentized_level_rms` vs `studentized`	14.8	36.7	−21.9	−23.9	−24.6
Base width: `studentized_full` vs `studentized_base`	20.8	23.0	−2.2	−2.1	−3.9

Table 8: Full base against median rebuild, mean over the six disease–design cells.

Yes, by a wide margin. With a constant or a recent-error scale, rebuilding from the median costs 22–32 points of loss, up to and after peaks alike and for both diseases. Most of this is the width: the median rebuild that keeps the CMU forecast's current width (`studentized_base`) loses only 2 points against its full-base twin, 5 for flu and none for COVID, and 14–25 points less than the median rebuilds with a constant or lagging residual scale. The weekly width of the CMU forecast is thus its main information beyond the median; its quantile shape adds about 5 points for flu. Starting all levels at the median instead of at normal quantiles costs another 1–3 points of loss and 4–16 points of calibration error.

Plots. Each pair is two bars of `multiqt_feedback_bars.pdf` (Fixed MultiQT and the groups "Studentized median" and "Studentized full base"), for every design: pooled on pp. 1 and 7, by location on pp. 2–6 and 8–12.

4.7 Expert weighting

Mixture	Loss			CE	
	overall	up to peak	after peak	overall	after peak
Rate grid, 12 rates $\kappa \leq .45$					
`uniform`	19.4	6.9	13.4	−33.2	−5.4
`weighted`	8.7	−1.9	11.1	−36.5	3.7
`weighted_common`	4.4	−3.4	7.1	−38.5	2.8
`shared`	12.4	6.6	16.1	−35.0	0.5
`shared_common`	8.0	3.3	10.6	−35.2	0.2
`shared_outer`	12.7	6.6	16.4	−37.0	0.8
Weekly starts					
`restart_persistent`	11.9	0.2	10.2	−44.7	0.3
`restart_uniform`	7.2	−0.6	7.1	−41.2	−2.8
`restart_weighted`	8.2	0.8	8.3	−36.9	−2.2
`restart_recent`	1.9	−0.4	2.0	−18.0	−6.4
`restart_recent_dual`	10.3	2.0	8.3	−20.0	−11.6
Score bank					
`score_equal`	6.4	−0.3	8.0	−35.8	−0.1
`score_learned`	6.6	−0.9	11.6	−28.1	−2.1

Table 9: Mixtures: pooled change versus the base (%), mean over the six disease–design cells. Weights are described for the saved COVID runs at $h = 2$, delay 0 (the five plotted locations).

Rate grid: which learning rates win. Learned weights beat equal weights clearly (`weighted` +8.7%, `uniform` +19.4%): equal weights give the fast rates as much weight as the slow ones. The best grid mixture has one weight vector for all levels and no sharing (`weighted_common`, +4.4%, −3.4% up to peaks, calibration error −39%); per-level weights add noise. Sharing costs about 4 points: it keeps $\gamma = 1/16$ of the mass spread over all twelve rates, a share of the equal-weight mixture. The outer MultiQT adds 0.3 points. Without sharing, the weight concentrates on two or three rates (effective number 2.8–3.4 of 12) and the rate depends on the location: around $\kappa = .03$–$.05$ in California, Texas and the US, but on the base ($\kappa = 0$: 32% and 52% of the weight) in New York and Pennsylvania. The largest rates gain weight only at upper levels (.3 and .45 together 9%; .45 the top expert in 6% of weeks), so extending the grid beyond .3 changes little.

Weekly starts: the age of the corrections. `restart_recent` is the best mixture (+1.9%; COVID −0.4% to +0.8%, flu +3% to +4%) and the only one that also improves after-peak calibration clearly for flu (−9% to −13%). Its learned weights barely move (effective number 7.8 of 8): it acts as an equal average of the eight youngest copies, so its gain comes from the age limit, not from learning. The two-rate version (`restart_recent_dual`, .03 and .3) splits its weight almost evenly between slow and fast copies at every level (47–50% on the fast ones; effective number 15 of 16). It therefore mixes in the fast correction: more loss (+10.3%; flu +13% to +14%) but the best after-peak calibration (−11.6%; flu −16% to −20%). Over all starts, equal and learned weights behave alike (+7.2%, +8.2%); the learned weight spreads over copies of every age, 29% on copies older than 32 weeks. The first start alone (`restart_persistent`) loses most (+11.9%). The weights of the weekly starts barely move because their losses share a large common part: the between-copy variance is about 2% of the weighted mean squared loss that sets the rate (Section 3.5.3.3); the weighted variance of the score mixture was not tried for them.

Score bank. Equal and learned weights perform alike (+6.4%, +6.6%); learned weights calibrate less (−28% against −36%). The base and Leaky carry most of the learned weight (0.22–0.32 and 0.25–0.33), with about two members dominating any week (effective number 2 of 5). Neither mixture beats its own member `sqrt_constant` (+0.8%).

Mixtures against a single fixed rate. Tables 10 (overall) and 11 (after peaks) compare two mixtures with Fixed MultiQT on the rate grid of script 02, in each disease–design cell. A fixed rate beats a mixture if both its loss and its calibration error are at most the mixture's.

		`weighted_common`			`restart_recent`		
Disease	Design	Loss	CE	Beaten by: κ (loss, CE)	Loss	CE	Beaten by: κ (loss, CE)
COVID	delay 0	3.4	−16.7	—	0.8	−16.4	—
	delay 1	3.9	−45.5	.02 (3.3, −47.2)	−0.4	−24.1	—
	re-run	4.6	−54.6	.015 (2.4, −58.2)	0.4	−23.7	—
Flu	delay 0	3.3	−28.8	.01 (1.2, −32.9)	4.0	−9.1	.01 (1.2, −32.9)
	delay 1	2.6	−40.1	.01 (1.5, −41.9)	3.1	−18.0	.01 (1.5, −41.9)
	re-run	8.3	−45.3	.015 (3.0, −47.2)	3.7	−16.9	.01 (1.7, −41.0)

Table 10: Best rate-grid mixture and most recent weekly starts against single fixed rates: pooled overall change versus the base (%). Beaten by: the rate $\kappa > 0$ of the grid with both loss and CE at most the mixture's (the lowest loss if several); —: no such rate.

The best rate-grid mixture, `weighted_common`, is beaten by a single rate between .01 and .02 in five of the six cells. Only for COVID at delay 0 is it ahead: a fixed rate needs +6.3% loss

($\kappa = .03$) for its calibration error. The other rate-grid mixtures (`uniform`, `weighted`, `shared`, `shared_common`) are beaten in all six cells, the score mixtures in all but COVID at delay 0. The beating rate is picked in hindsight, but it lies between .01 and .02 in every cell, and $\kappa = .015$ throughout would have matched the mixture's calibration at less loss (means +2.7% and −38.8% against +4.4% and −38.5%). Mixing rates thus protects against a badly selected rate (`fixed`, +16.2%), but does not improve on a small fixed one.

		`weighted_common`			`restart_recent`		
Disease	Design	Loss	CE	Beaten by: κ (loss, CE)	Loss	CE	Beaten by: κ (loss, CE)
COVID	delay 0	7.3	−1.7	.01 (2.4, −4.8)	1.4	−3.8	—
	delay 1	11.7	6.1	.01 (4.3, 2.4)	1.3	−0.5	—
	re-run	12.9	8.0	.01 (5.4, 6.0)	1.8	−2.3	—
Flu	delay 0	2.4	0.9	.01 (0.5, −0.5)	3.2	−9.8	—
	delay 1	1.0	1.2	.01 (0.0, −0.4)	1.5	−9.0	—
	re-run	7.0	2.2	.01 (0.7, 0.4)	2.9	−13.0	—

Table 11: As Table 10, after peaks (weeks 1 to 8).

After peaks (Table 11), the smallest rate, $\kappa = .01$, beats the rate mixture in every cell, and the base itself does in five: the rates above .01 that the mixture weighs carry the correction learned during the rise past the peak (Section 4.2). `restart_recent` is beaten by no rate after peaks. For COVID it loses less than every rate (+1.3% to +1.8%) and changes the calibration error little; for flu it reduces the calibration error by 9 to 13% at +1.5% to +3.2% loss, while no fixed rate gets below −1.5% there. Overall, no rate beats it for COVID either: its loss (−0.4% to +0.8%) is below that of every rate, at 16 to 24% less calibration error than the base. For flu, $\kappa = .01$ beats it overall in every design: it trades calibration over the season for calibration after peaks. Its weights stay almost uniform, so these gains come from the eight-week memory of its copies, not from learning which copy to trust. Of the single trackers, REF-op is beaten by no fixed rate overall in any cell, and `sqrt_constant` only for COVID at delay 1 and in the re-run.

Plots. Performance: `multiqt_feedback_bars.pdf`, groups "Rate-grid experts", "Weekly starts: immediate admission" and "Score-based experts" (pooled: pp. 1 and 7). Weights: `multiqt_weight_timelines/`, per disease, horizon and design: the rate grid on pp. 1, 5, 9, 13 and 17 (columns "Equal weights", "Learned, no sharing" and "Learned + sharing"; the rows "All levels" show their versions with common weights), the score experts on pp. 2, 6, 10, 14 and 18, the weekly starts on pp. 3, 7, 11, 15 and 19 (`restart_recent` in the column "Most recent starts: learned + sharing") and the two-rate weekly starts on pp. 4, 8, 12, 16 and 20. Their forecasts: the pages "Rate-grid experts", "Weekly starts: immediate admission" and "Score-based experts" of `multiqt_method_forecasts/` (the third to fifth of each location's seven). The effective numbers and mean weights quoted above summarize the weights for COVID, $h = 2$, delay 0 (`covid_horizon2_delay0.pdf`); they are not plotted. Neither are Tables 10 and 11; the fixed-rate curves they use are those of `multiqt_phase_rate_tradeoffs.pdf`, pp. 1–3, and the mixtures are bars of `multiqt_feedback_bars.pdf`, pp. 1 and 7.

4.8 Locations

Disease	Location	fixed	fixed _log	sqrt _const.	ref _op	stud. level_rms	weighted _common	restart _recent	restart _dual	score _equal
COVID	52 pooled	19.4	19.1	1.2	0.0	13.3	3.7	0.4	6.8	5.5
	US	34.8	23.6	4.3	1.6	20.2	15.1	−1.4	5.8	10.0
	California	19.0	16.0	2.4	−4.5	26.6	9.9	−8.7	−11.4	3.2
	Texas	22.8	33.8	4.8	−2.1	32.5	10.7	−5.9	−6.7	7.1
	New York	30.7	14.6	1.9	−0.4	8.5	3.9	1.2	13.5	5.6
	Pennsylvania	29.4	29.2	4.2	3.4	19.4	6.0	2.6	11.8	12.0
Flu	52 pooled	12.4	43.7	0.5	1.0	15.3	4.8	3.7	14.1	7.1
	US	13.0	60.3	−1.4	−1.2	18.8	15.4	1.7	10.4	4.6
	California	15.9	23.5	−7.9	−6.6	9.9	14.1	−1.8	5.3	1.3
	Texas	10.3	24.1	0.6	0.6	4.8	2.4	2.0	9.8	4.2
	New York	9.0	28.1	−2.7	−0.8	29.2	2.1	0.0	2.9	8.1
	Pennsylvania	16.0	61.8	−1.7	0.4	29.2	4.6	6.1	21.9	11.4

Table 12: Overall loss change versus the base (%) by location, mean over the three feedback designs (pooled: the 52 locations without the US).

- **COVID splits the plotted locations.** California and Texas gain most up to peaks (Fixed −26% and −4%, `restart_recent` −16% and −15%), and several methods beat the base overall there (`restart_recent` −8.7% and −5.9%, the two-rate starts −11.4% and −6.7%, REF-op −4.5% and −2.1%); they also lose most after peaks (Fixed +48% and +32%). New York, Pennsylvania and the US lose up to peaks as well (Fixed +28% to +59%); there only the Delphi trackers and `restart_recent` stay within about 4% of the base.
- **Flu** gains up to peaks in all plotted locations except Texas, and the after-peak losses decide: the `log1p` trackers lose most (after peaks +177% in the US, +157% in Pennsylvania), while `sqrt_constant` and REF-op beat the base in California (−7.9%, −6.6%), New York and the US.
- **Calibration** improves strongly for flu everywhere (Fixed −35% to −64% in the plotted locations). For COVID the large plotted locations gain little (Fixed between +3% and −17%, pooled −45%), and REF-op even worsens their calibration error (+7% to +19% in the US, California, Texas and Pennsylvania).
- Pooled results thus hide large differences: for COVID the ranking of methods in California and Texas is not the one in New York and Pennsylvania. The US, as the sum of the states, resembles New York and Pennsylvania for COVID.

Plots. `multiqt_feedback_bars.pdf`, pp. 2–6 (COVID) and 8–12 (flu), every method per location; the location pages of `multiqt_evaluation_tree.pdf` (pp. 2–6 and 8–12) and of `multiqt_peak_reasoning_tradeoff.pdf` (pp. 3–7 and 10–14); the fixed rates per location in `multiqt_error_tradeoff.pdf`, pp. 2–7.

References

[1] Tiffany Ding, Isaac Gibbs, and Ryan J. Tibshirani. *Calibrated Multi-Level Quantile Forecasting.* arXiv:2512.23671v1, 2025. MultiQT paper.

[2] Anastasios N. Angelopoulos, Emmanuel J. Candès, and Ryan J. Tibshirani. *Conformal PID Control for Time Series Prediction.* Advances in Neural Information Processing Systems 36, 2023. NeurIPS paper.

[3] Isaac Gibbs and Emmanuel J. Candès. *Conformal Inference for Online Prediction with Arbitrary Distribution Shifts*. Journal of Machine Learning Research 25, 2024. JMLR paper.

[4] CMU Delphi. *Delphi hub forecasting: calibration experiments and tracker code*. Pinned source revision `9abebcfb`, inspected 9–10 November 2026. E09: constant learning rate and revision delay; E15: median shift and width change; tracker implementation.